

Análise e Desenvolvimento de Sistemas

Gerenciamento Ágil de Projetos

Design Thinking aplicado a software

Período: **08/05/2026** a **03/07/2026**

Em grupo | Todos os grupos abordam os 5 temas | 8 sprints + entrega final

Problema real escolhido pelo grupo

O grupo aplicou as cinco etapas do Design Thinking ao desenvolvimento do Scrunic, uma plataforma de gestão de tarefas com quadro Kanban, construída com Node.js e MySQL no back-end e React com TypeScript no front-end. O objetivo desta sprint foi sair do conceito estudado na Sprint 3 e aplicar o processo completo a um problema real do próprio grupo, do primeiro contato com o usuário até um protótipo testável.

1. Empatia

A equipe entrevistou seis colegas que organizavam tarefas de projetos acadêmicos em anotações soltas, planilhas e mensagens de WhatsApp. Em vez de perguntar diretamente "o que você gostaria em um app de tarefas", as entrevistas focaram em pedir para a pessoa descrever como organizava o último trabalho em grupo que fez. A descoberta central: as pessoas não sabiam, a qualquer momento, o que estava pendente, em andamento ou concluído no grupo — e descobriam isso só quando alguém perguntava diretamente, geralmente perto do prazo de entrega.

2. Definição

Problema definido: "Como podemos dar ao grupo uma visão única e atualizada do que cada pessoa está fazendo, sem depender de mensagens espalhadas?"

Essa formulação foi escolhida entre três alternativas discutidas pelo grupo, descartando versões mais restritas como "como podemos lembrar as pessoas de suas tarefas", que focava em notificação e não no problema real de visibilidade coletiva identificado nas entrevistas.

3. Ideação

Entre as ideias levantadas em uma sessão de brainstorming de 20 minutos estavam um bot de WhatsApp que respondia com o status das tarefas, uma planilha compartilhada com cores por status, um quadro físico fotografado e compartilhado diariamente, e um quadro visual estilo Kanban dentro de um aplicativo próprio. O quadro visual foi a ideia mais votada por ser a mais simples de entender de relance e por não depender de disciplina manual de atualização, como exigiria a planilha ou o quadro físico.

4. Prototipação: wireframe de baixa fidelidade

A equipe esboçou a tela principal do Scrunic em baixa fidelidade, representando apenas a estrutura e a posição dos elementos, sem preocupação com cores ou estilo final. O objetivo do wireframe não era definir o design visual, mas validar se a organização em colunas resolvia o problema de visibilidade identificado na etapa de definição.

```
+-----+
|  SCRUNIC                                [+ Nova Tarefa] |
+-----+
|  A FAZER          |  EM ANDAMENTO    |  EM REVISAO  | PRONTO  |
|  [ ] Tarefa 1    |  [ ] Tarefa 4    |  [ ] Tarefa 6|[x]T8 |
|  [ ] Tarefa 2    |  [ ] Tarefa 5    |  [ ] Tarefa 7|[x]T9 |
|  [ ] Tarefa 3    |                    |              |
+-----+
```

O esboço já responde à pergunta de definição: cada coluna representa um status e cada cartão uma tarefa, dando visibilidade imediata do fluxo de trabalho do grupo. Propositamente, o wireframe não tem cores, ícones ou tipografia definida — qualquer decisão estética nesse momento distrairia o teste do que realmente importava: a estrutura resolve o problema?

5. Teste

O wireframe foi mostrado a três colegas fora do grupo de desenvolvimento, em sessões individuais de dez minutos, pedindo que descrevessem o que cada coluna significava sem explicação prévia. Dois deles pediram um contador de tarefas por coluna e um filtro por responsável — ajustes incorporados ao protótipo antes do início da implementação em React. O terceiro colega confundiu inicialmente as colunas "Em Revisão" e "Pronto", o que levou a equipe a renomear a última para "Concluído", eliminando a ambiguidade antes de qualquer linha de código ser escrita.

Ideia central: cada ajuste identificado no teste custou minutos de conversa nesta sprint — o mesmo ajuste, descoberto só depois do sistema pronto, custaria retrabalho de tela, banco de dados e testes automatizados.